

Práctica: Armado de un Workflow de Machine Learning con Airflow

Objetivo

Esta práctica tiene como objetivo principal comprender y aplicar los conceptos de **Machine Learning Operations (MLOps)** mediante la construcción de un pipeline en **Apache Airflow**.

Pre-requisitos

- Conocimientos básicos de Docker y Docker Compose.
- Conocimientos básicos de Python.

Materiales y Herramientas

- Editor de texto/IDE (VS Code, PyCharm, etc.).
 - Terminal/Línea de comandos.
 - Docker y Docker Compose
-

1. Introducción a Apache Airflow

Como ya sabrán, Apache Airflow es una plataforma de código abierto que permite **crear, programar y monitorizar flujos de trabajo (workflows) de manera programática**. En el contexto de Machine Learning (ML), Airflow se utiliza comúnmente para orquestar pipelines de datos complejos, desde la ingesta y el preprocesamiento hasta el entrenamiento del modelo y el despliegue.

Un flujo de trabajo en Airflow se define como un **DAG (Directed Acyclic Graph)**, que representa una colección de tareas con dependencias específicas que deben ejecutarse secuencialmente. Airflow ofrece una interfaz de usuario para visualizar los DAGs, monitorizar su estado y gestionar las ejecuciones.

Preparación:

- Descargar e instalar [Docker Desktop](#), que convenientemente trae Docker Compose
- Descargar el archivo [docker-compose.yaml](#)

- Configurar el directorio en el que trabajaremos corriendo los siguientes comandos

Shell

```
mkdir -p ./dags ./logs ./plugins ./config  
echo -e "AIRFLOW_UID=$(id -u)" > .env  
echo "_PIP_ADDITIONAL_REQUIREMENTS=pandas scikit-learn" >>  
.env
```

- Inicializar la base de datos

Shell

```
docker compose up airflow-init
```

- Desplegar el Scheduler, la API con la interfaz web para el usuario, el worker que ejecuta las tareas, y la base de datos.

Shell

```
docker compose up -d
```

Una vez levantados todos los servicios, deberíamos poder acceder a <http://localhost:8080> en donde encontraremos la interfaz web de Airflow. El usuario/contraseña por defecto son **airflow/airflow**

Pythonic DAGs

Usaremos la [TaskFlow API](#) para que nuestros DAGs sean mas *pythonicos*. Usualmente la comunicación entre tareas se maneja mediante XCom, que es un mecanismo de Airflow para intercambiar mensajes usando la base de metadatos.

TaskFlow API nos permite usar decoradores para definir DAGs y Tasks, ayudándonos a abstraernos de gran parte de esa lógica. Por debajo usa XCom, pero sintácticamente es idéntico a cualquier función de Python.

Como referencia, este es un ejemplo del mismo DAG escrito de manera tradicional y utilizando TaskFlow API

```
pythonic.py U X
p1 > dags > pythonic.py > ...
1 from airflow.decorators import dag, task
2
3 @dag(
4     dag_id='pythonic_dag',
5     description='DAG Pythonico para practicar conceptos básicos',
6 )
7 def dag_tutorial():
8
9     @task
10    def imprimir(mensaje):
11        print(mensaje)
12
13    @task
14    def generar_datos():
15        import random
16        return [random.randint(0, 100) for _ in range(1000)]
17
18    @task
19    def calcular_suma(l):
20        return sum(l)
21
22    imprimir(calcular_suma(generar_datos()))
23
24 dag_tutorial()
25

traditional.py U X
p1 > dags > traditional.py > ...
1 from airflow import DAG
2 from airflow.operators.python import PythonOperator
3
4
5 def _generar_datos(**context):
6     import random
7     return [random.randint(0, 100) for _ in range(1000)]
8
9
10 def _calcular_suma(**context):
11     datos = context['ti'].xcom_pull(task_ids='generar_datos')
12     return sum(datos)
13
14
15 def _imprimir(**context):
16     suma = context['ti'].xcom_pull(task_ids='calcular_suma')
17     print(suma)
18
19
20 with DAG(
21     dag_id='traditional_dag',
22     description='DAG tradicional para practicar conceptos básicos',
23 ) as dag:
24
25     generar_datos = PythonOperator(
26         task_id='generar_datos',
27         python_callable=_generar_datos,
28     )
29
30     calcular_suma = PythonOperator(
31         task_id='calcular_suma',
32         python_callable=_calcular_suma,
33     )
34
35     imprimir = PythonOperator(
36         task_id='imprimir',
37         python_callable=_imprimir,
38     )
39
40     generar_datos >> calcular_suma >> imprimir
41
```

2. Workflow de Machine Learning

Para este segundo ejercicio se propone armar un DAG que descargue el dataset de [Producción de Pozos de Gas y Petróleo No Convencional](#) y entrene dos modelos de regresión por Random Forest para estimar `prod_pet` y `prod_gas`.

Se provee para esto el siguiente esqueleto a ser completado

Python

```
from airflow.decorators import dag, task
from airflow.operators.empty import EmptyOperator
```

```
DATASET_DOWNLOAD_URL =
```

```
"http://datos.energia.gob.ar/dataset/c846e79c-026c-4040-897
f-1ad3543b407c/resource/b5b58cdc-9e07-41f9-b392-fb9ec68b072"
```

```
5/download/produccion-de-pozos-de-gas-y-petroleo-no-convencional.csv"
```

```
@dag(
    dag_id='ml_pipeline',
    description='Pipeline de Machine Learning con Airflow',
)
def ml_pipeline():
    start = EmptyOperator(
        task_id='start',
    )

    @task
    def download_dataset(url, save_path):
        """Descarga un dataset CSV desde una URL y lo guarda en
        disco.
        Args: url (str) - URL de descarga, save_path (str) -
        ruta local del archivo.
        Retorna: la ruta del archivo guardado.
        """
        return

    @task
    def preprocess(csv_path):
        """Lee el CSV, selecciona las columnas relevantes y
        codifica
        la columna 'tipoextraccion' como enteros.
        Args: csv_path (str) - ruta al archivo CSV.
        Retorna: dict con nombres de columnas como claves y
        listas de valores.
        """
        return
```

```
@task
def train_model(data, target, features, model_path):
    """Entrena un RandomForestRegressor para predecir el
    target indicado.

    Guarda el modelo entrenado en la ruta especificada en
    formato pickle.

    Args: data (dict) - dataset preprocesado, target (str)
    - columna a predecir,
           features (list) - columnas de entrada, model_path
    (str) - ruta donde guardar el modelo.

    Retorna: dict con metadata del modelo (target,
    features, ruta, etc.).
    """

    from sklearn.ensemble import RandomForestRegressor
    return
```

```
@task
def predict(model_info, sample):
    """Carga un modelo entrenado desde disco y predice el
    valor del target.

    Args: model_info (dict) - metadata del modelo
    incluyendo 'model_path',
           sample (list) - dato de entrada para predecir.

    Retorna: dict con model_info más 'sample' y
    'prediction'.
    """

    return
```

```
@task
def show_results(results):
    """Imprime un resumen de los resultados de predicción
    del modelo.
```

```
    Args: results (dict) - contiene target, features,  
          n_samples, random_sample, prediction.
```

```
    """
```

```
    return
```

```
    # Completar con la secuencia de tasks
```

```
ml_pipeline()
```

Los resultados pueden ser visualizados en los logs de las tareas llamadas “show_results”

The screenshot shows the Apache Airflow web interface. On the left, the DAG 'ml_pipeline' is visualized, showing a sequence of tasks: 'licit', 'licit_1', 'show_results', and 'show_results_1'. The 'show_results' task is highlighted with a blue box. On the right, the logs for the 'show_results' task are displayed. The logs show the task's execution details, including the operator, date, duration, and version. The log messages indicate that the task was successful and returned a prediction of 56.33.

Operador	Fecha Inicial	Fecha Final	Duración	Versión del Dag
@task	2026-03-06 11:15:16	2026-03-06 11:15:17	00:00:00.360	v5

Logs

Todos los niveles de log

Log message source details sources=["/opt/airflow/logs/dag_id=ml_pipeline/run_id>manual__2026-03-06T14:13:21.758477"]

```
2 [2026-03-06 11:15:17] INFO - DAG bundles loaded: dags-folder, example_dags
3 [2026-03-06 11:15:17] INFO - Filling up the DagBag from /opt/airflow/dags/ml_pipeline.py
4 [2026-03-06 11:15:17] WARNING - The 'airflow.decorators.dag' attribute is deprecated. Please use 'airflow.sdk
5 [2026-03-06 11:15:17] WARNING - The 'airflow.decorators.task' attribute is deprecated. Please use 'airflow.sd
6 [2026-03-06 11:15:17] WARNING - The 'airflow.decorators.empty' attribute is deprecated. Please use 'airflow.sd
7 [2026-03-06 11:15:17] INFO - Task instance is in running state
8 [2026-03-06 11:15:17] INFO - Previous state of the Task instance: TaskInstanceState.QUEUED
9 [2026-03-06 11:15:17] INFO - Current task name: show_results
10 [2026-03-06 11:15:17] INFO - Dag name: ml_pipeline
11 [2026-03-06 11:15:17] INFO -
12 [2026-03-06 11:15:17] INFO - =====
13 [2026-03-06 11:15:17] INFO - Predicción de: prod_gas
14 [2026-03-06 11:15:17] INFO - Features: ['prod_pet', 'prod_agua', 'tef', 'profundidad', 'tipoextraccion']
15 [2026-03-06 11:15:17] INFO - Muestras utilizadas: 390659
16 [2026-03-06 11:15:17] INFO - Datos: [250.0, 100.0, 15.0, 3000.0, 1]
17 [2026-03-06 11:15:17] INFO - Predicción: 56.33
18 [2026-03-06 11:15:17] INFO - Done. Returned value was: None
19 [2026-03-06 11:15:17] INFO - =====
20 [2026-03-06 11:15:17] INFO -
21 [2026-03-06 11:15:17] INFO - Task instance in success state
22 [2026-03-06 11:15:17] INFO - Previous state of the Task instance: TaskInstanceState.RUNNING
23 [2026-03-06 11:15:17] INFO - Task operator: <Task(PythonDecoratedOperator): show_results>
```

3. Extendiendo el Workflow

Ahora queremos extender el DAG anterior. Para esto proponemos:

Modificar el DAG para que se le indique un rango de fechas y luego utilice sólo datos dentro de ese rango.

Python

```
# ...

@dag(
    dag_id='ml_pipeline',
    description='Pipeline de Machine Learning con Airflow',
    params={
        'date_from': Param(
            default=None,
            type=['null', 'string'],
            description='Fecha inicio del rango (YYYY-MM-DD). Dejar vacío para no filtrar.',
        ),
        'date_to': Param(
            default=None,
            type=['null', 'string'],
            description='Fecha fin del rango (YYYY-MM-DD). Dejar vacío para no filtrar.',
        ),
    },
)
def ml_pipeline():
    # ...

    @task
    def preprocess(csv_path):
        params = context['params']
        date_from = params.get('date_from')
        date_to = params.get('date_to')
        return
```

...

Preguntas disparadoras

- Supongamos que nuestro foco es la iteración. Diariamente entrenamos múltiples modelos. ¿Cómo haríamos para registrar y comparar las métricas que producen cada uno?
- Si cambiáramos el número de estimadores, ¿cómo sabríamos cuál dió mejor resultado? ¿en algún lugar tenemos registro de los hiperparametros usados en cada ejecución?
- El DAG guarda los modelos en archivos de formato pickle, pisando siempre la última versión. ¿Cómo podrías volver a una versión anterior del modelo si la nueva resultó ser peor?
- Supongamos que tuvimos un buen resultado en un entrenamiento, ¿tenemos toda la información necesaria para reproducirlo otra vez?
- ¿Cómo procedemos, si por el contrario, recibimos quejas respecto al comportamiento del modelo?